

Spec-Driven Development

Ablauf & Spickzettel für KI-gestütztes Programmieren

Dieser Spickzettel beschreibt eine feste, aufeinander abgestimmte Abfolge von Arbeitsschritten („Liturgie“), mit der sich ein Software-Projekt zuverlässig mit einem KI-Coding-Agenten umsetzen lässt. Wichtiger als der genaue Wortlaut der Prompts ist die Idee dahinter:

LEITPRINZIP — GRUNDIDEE VOR WORTLAUT

Jeder Schritt erklärt zuerst die Idee (worum es geht und warum), danach folgt ein fertiger Prompt als Werkzeug. Der Prompt ist unverzichtbar in ausformulierter Form — aber kein Gesetz. Verstehe, worum es geht, und formuliere dort um, wo dein Projekt es verlangt.

Die Artefakte auf einen Blick

SDD trennt verschiedene Ebenen sauber in eigene Dateien. Diese Trennung ist der Kern der Methode:

Datei	Rolle	Geltung
<code>constitution.md</code>	Unveränderliche Prinzipien (optional)	gilt für alles
<code>spec.md</code>	WAS gebaut wird — Anforderungen	pro Projekt / Feature
<code>plan.md</code>	WIE gebaut wird — Architektur, Bibliotheken	pro Projekt
<code>tasks.md</code>	Die SCHRITTE — klein, einzeln testbar	pro Projekt
<code>STATUS.md</code>	ZUSTAND bei Unterbrechung — Übergabe	bei Bedarf

MENTALES MODELL — DATEIEN SIND DAS GEDÄCHTNIS

Die .md-Dateien liegen auf der Festplatte und überleben jede Session — sie sind das Langzeitgedächtnis. Das Kontextfenster des Modells ist nur flüchtiger Arbeitsspeicher (RAM). Daraus folgt die wichtigste Arbeitsregel: nicht alles in eine Session zwingen. Saubere Zwischenstände per Git sichern, neue Session starten — die Dateien tragen den Zustand weiter.

Farb-Legende in den Prompts:

- Grün** = von dir frei formulierter Inhalt (z. B. deine Projektidee).
- Rot** = Platzhalter/Stellen, die du an dein Projekt anpasst.

Schritt 1 — Spec anstoßen (Metaprompt)

DIE IDEE

Bevor eine Zeile Spezifikation entsteht, zwingt dieser Schritt das Modell, deine vage Idee durch gezielte Rückfragen vollständig auszuleuchten. Das Modell soll keine Spec aus Annahmen bauen, sondern aus geklärten Anforderungen. Der eigentliche Wert liegt in den Fragen, die es dir stellt.

Du agierst ab sofort als sehr erfahrener Senior Software Architect und Product Manager. Mein Ziel ist es, nach den Prinzipien des Spec-Driven Development (SDD) zu arbeiten. Am Ende sollst du mir eine wasserdichte, maschinenlesbare Markdown-Spezifikation (spec.md) für ein Feature/Projekt schreiben, die ich direkt an einen KI-Coding-Agenten übergeben kann.

Hier ist meine noch sehr vage Idee:

[deine Projektidee in eigenen Worten – Beispiel:]

Oberstufenschüler im Fach Physik haben oft Probleme mit der Einheiten-Rechnung. Eine Web-App (HTML/CSS/JavaScript, als statische Seite gehostet) soll ihnen das Einüben der Umrechnung von SI-Grundeinheiten zu zusammengesetzten Größen erleichtern, mit Aufgaben auf mehreren Niveaustufen und gestuften Hilfen.

WICHTIGE REGEL: Schreibe die Spezifikation noch NICHT. Bevor wir sie erstellen, musst du alle Unklarheiten beseitigen. Stelle mir eine strukturierte, nummerierte Liste der 10 bis 14 wichtigsten Fragen.

Gruppieren sie in folgende Kategorien:

1. Didaktische Vollständigkeit / pädagogischer Mehrwert
2. User-Interface-Elemente und Layout
3. Technische Umsetzbarkeit und Grenzen
4. Art der Datenspeicherung / Daten-Repräsentation
5. Testing & Akzeptanzkriterien

Stelle präzise, technische Fragen. Biete bei komplexen Punkten direkt 2 bis 3 Best-Practice-Optionen (A, B, C) an, aus denen ich wählen kann.

Warte nach deinen Fragen auf meine Antworten. Wenn meine Antworten weitere Lücken aufwerfen, hake nach. Deine Einschätzung "keine Lücken mehr" ist eine Heuristik, keine Garantie – bleibe kritisch. Erst wenn du hinreichend sicher bist, sage ich: "Erstelle jetzt die spec.md".

Anpassen: Die fünf Kategorien (rot) sind hier didaktisch geprägt — für ein nicht-pädagogisches Projekt ersetzt du sie durch passende Bereiche (z. B. Domänenlogik, Performance, Sicherheit).

Schritt 2 — Spec-Review (am besten mit einem anderen Modell)

DIE IDEE

Ein zweites Modell findet Lücken, die das erste übersieht — frische Augen, anderer Trainings-Bias. Es geht ums Härten der Spec, nicht um Code.

Lies die @spec.md. Du bist erfahrener Senior-Entwickler. Prüfe die Spezifikation auf Architektur- und Logiklücken, BEVOR wir Code schreiben. Stelle mir gezielte Rückfragen zu allem, was unklar oder widersprüchlich ist. Schreibe noch keinen Code.

Schritt 3 — plan.md erstellen

DIE IDEE

Übersetzt das WAS der Spec in ein WIE: Projektstruktur, Datenfluss, gewählte Bibliotheken — jeweils mit kurzer Begründung. Trennt Architekturentscheidungen von der eigentlichen Umsetzung.

Erstelle aus @spec.md einen technischen Plan und speichere ihn als plan.md: Projektstruktur (Dateien/Module), Datenfluss, gewählte Bibliotheken und kurze Begründungen. Halte dich an die Constraints in spec.md. Noch kein Implementierungscode.

Schritt 4 — tasks.md erstellen

DIE IDEE

Zerlegt den Plan in kleine, einzeln testbare Schritte. Die Granularität ist deine wichtigste Versicherung gegen Endlosschleifen und Kontext-Überlauf: Jeder Task braucht ein eindeutiges „Fertig“-Signal — sonst weiß das Modell nicht, wann es aufhören soll.

Leite aus @plan.md eine Aufgabenliste ab und speichere sie als tasks.md: kleine, einzeln testbare Schritte in sinnvoller Reihenfolge. Notiere zu jedem Task: (a) ein Satz Ziel, (b) das messbare Fertig-Kriterium (welcher Test muss grün sein?), (c) die dafür relevanten Dateien. Lege vor jeden Task eine Checkbox [] zum Abhaken.

Schritt 5 — Tests zuerst (TDD)

DIE IDEE

Die Tests sind die ausführbare Form deiner Akzeptanzkriterien. Sie schlagen zuerst fehl (rot) und definieren damit objektiv und unbestechlich, wann ein Task erledigt ist.

Wir arbeiten @tasks.md Schritt für Schritt ab. Beginne mit den Tests. Schreibe auf Basis der Akzeptanzkriterien in @spec.md solche Tests, die das Verhalten prüfen und zunächst fehlschlagen. Erkläre kurz, was jeder Test prüft. Schreibe noch keinen Implementierungscode.

Schritt 6 — Implementieren

DIE IDEE

Jetzt Code, bis die Tests grün sind. Nach jedem grünen Task committen — das ist dein Speicherstand, von dem aus eine neue Session sauber starten kann. (Beachte die Schutzmechanismen weiter unten.)

Implementiere jetzt den Code, bis alle Tests grün sind. Führe die Tests aus und zeig mir das Ergebnis. Schlagen Tests fehl, korrigiere und wiederhole. Hake erledigte Tasks in tasks.md ab. Mache nach jedem grünen Task einen Git-Commit mit aussagekräftiger Nachricht.

Schritt 7 — Wiederaufnahme nach Unterbrechung (Handoff)

DIE IDEE

Ein frischer Agent würde sonst das ganze Repository durchlesen und das halbe Budget verbrennen, bevor er etwas tut. Stattdessen übergibst du ihm den Zustand fertig verdaut — das ist Context Engineering in Reinform: minimal hinreichender Kontext statt Repo-Dump. Zwei Mittel: eine kurze STATUS.md und die ausdrückliche Anweisung, NUR bestimmte Dateien zu lesen.

Pflege am Ende jeder Session eine STATUS.md nach diesem Muster:

VORLAGE — STATUS.md

```
# Projektstatus
## Erledigt:      Tasks 1-4 (Tests grün, committed: a1b2c3d)
## In Arbeit:     Task 5 – Formel-Parser, Tests noch rot
## Als Nächstes:  Task 6 – Hilfestufen-Logik
## Bekannte Probleme: KaTeX-Rendering zeigt Rohtext (s. Bug-Log)
## Relevante Dateien für Task 5/6: src/parser.js, src/hints.js, tests/parser.test.js
```

Der Wiederaufnahme-Prompt zeigt dann gezielt darauf — nicht aufs ganze Repo:

Wir führen ein unterbrochenes Projekt weiter. Lies NUR @STATUS.md und @tasks.md, um dich zu orientieren – lies NICHT das gesamte Repository. Öffne ausschließlich die in STATUS.md genannten relevanten Dateien. Der nächste offene Task ist Nr. 5. Fang dort an. Wenn dir Information fehlt, frag nach, statt blind weitere Dateien einzulesen.

Faustregel: Eine Allowlist („öffne nur diese Dateien“) ist immer stärker als eine Denylist („ignoriere node_modules“). Alternativ orientiert auch `git log --oneline -10` plus der letzte Diff einen Agenten in wenigen hundert Tokens.

Schritt 8 — Bugfixing

DIE IDEE

Ein dünner Bug-Satz ist „garbage in — garbage out“. Ein guter Bug-Report ist selbst ein Stück Context Engineering: Er liefert Soll, Ist, Reproduktion und Vermutung mit. Und es gilt: erst diagnostizieren, dann fixen — kein blindes Herumpatchen.

Es liegt ein Bug vor. Beschreibung nach folgendem Schema:

1. Soll-Verhalten: [was sollte passieren]
2. Ist-Verhalten: [was passiert stattdessen, mit Beispiel]
3. Reproduktion: [welche Eingabe / Seite / Aktion löst es aus]
4. Vermuteter Ort: [wo du den Fehler tippst – ohne Fix vorzugeben]
5. Schon probiert: [was du ausgeschlossen hast]
6. Constraints: [was nicht kaputtgehen darf]

Vorgehen: Nenne mir zuerst die vermutete Ursache und deinen Plan.
Schreibe, wenn möglich, zuerst einen Test, der den Bug reproduziert
(und fehlschlägt), und behebe ihn dann. Erst dann committen.

BEISPIEL — WARUM KONTEXT ZÄHLT

Im Symptomtext deines Formel-Bugs stehen *doppelte* geschweifte Klammern: $\frac{\{\{1\}\}\{\{2\}\}}{\{\{3\}\}\{\{4\}\}}$. Das ist ein typisches Escape-Artefakt aus f-Strings oder Template-Engines (wo `\{\{ \}` für eine einzelne Klammer steht). Die Ursache liegt damit vermutlich nicht im Browser-Rendering, sondern eine Stufe früher beim Erzeugen des Formel-Strings — eine Spur, die ein dünner Prompt nie mitgeliefert hätte.

ENDLOSSCHLEIFEN ABFANGEN (CIRCUIT-BREAKER)

Wenn sich ein Modell beim Coden „aufhängt“, ist das fast immer ein Symptom — und strukturell abfangbar:

- **Circuit-Breaker.** Nimm einen Stopp-Befehl ins Arbeits-Prompt auf — der wirksamste Einzeltrick:

```
"Wenn derselbe Test nach deinem Fix zweimal hintereinander fehlschlägt, STOPP. Kein dritter Versuch. Fasse stattdessen zusammen, was du probiert hast, nenne deine zwei wahrscheinlichsten Ursachen-Hypothesen und frag mich, bevor du weitermachst."
```

- **Loop = Signal für eine schlechte Spec.** Modelle drehen sich meist dort im Kreis, wo eine Anforderung widersprüchlich oder unterspezifiziert ist. Repariere dann die Spec, nicht den Code.
- **Rollen trennen.** Ein billiges, schnelles Modell ist gut für mechanische Umsetzung klar spezifizierter Tasks, aber schlecht im Debugging kniffliger Fehler — genau dort entstehen Schleifen. Lass es keine subtilen Bugs jagen.
- **Kontext zurücksetzen.** Eine Schleife vergiftet ihren eigenen Kontext (Fixierung auf Fehlversuche). Oft ist der schnellste Ausweg: Session killen, Code + fehlschlagenden Test + saubere Symptombeschreibung nehmen, frische Session starten.
- **Manuell eingreifen.** Sobald dieselbe Datei zum dritten Mal editiert oder ein identischer Tool-Call wiederholt wird: unterbrechen. Nicht auf Selbstheilung hoffen und acht weitere Runden bezahlen.

KONTEXTFENSTER & SESSION-MANAGEMENT

„66 % Kontext verbraucht, erst 4 von 12 Tasks fertig“ — die Panik kommt aus einem falschen Modell: dass die 12 Tasks ein Marathon in einer Session sind. Sind sie nicht.

- **Eine Session ≈ 1–3 Tasks.** Nach ein paar Tasks committen, dann neue Session für die nächsten. Die neue Session liest nur, was sie braucht.
- **Git-Commit = Speicherstand.** Jeder Commit ist ein Speicherstand — gleichzeitig deine Absicherung gegen Schleifen.
- **tasks.md abhaken.** Erledigte Tasks markieren; so weiß die nächste Session sofort, wo es weitergeht.
- **Steuerdateien schlank halten.** Aufgeblähte spec/plan/tasks zahlen bei jedem Session-Start die Re-Read-Steuer.

Optional: constitution.md — brauchst du sie?

Die constitution.md enthält die unveränderlichen Prinzipien, die für alles gelten, was du baust (z. B. „immer TDD“, „vanilla JS, kein Build-Step“, „barrierefrei“). Sie ist die Verfassung; die spec.md ist der Bauplan für ein konkretes Projekt. Ob du sie brauchst, entscheidet diese Verzweigungsregel:

VERZWEIGUNGSREGEL

- **Weglassen, wenn:** Einzelprojekt, eine Person, in wenigen Sessions fertig (wie dein Einheiten-Rechner) — keine eigene constitution. Pack die drei bis fünf harten Regeln oben in die spec.md unter „Constraints“.
- **Einsetzen, wenn:** ein Projekt über mehrere Specs/Features wächst und sich Invarianten wiederholen; mehrere Personen oder Agenten daran arbeiten; oder ein Agent dieselbe Regel ständig verletzt und du sie ihm „als Gesetz“ vorlegen willst.
- **Faustregel:** Wenn du dieselben Constraints in jede neue spec.md kopierst — dieses Copy-Paste ist deine constitution. Zieh sie heraus.
- **Sonderfall Lehre:** Im Unterricht kannst du sie als Qualitäts-Template mitgeben, das alle Schülerprojekte erben. Kurz halten: 5–10 knappe, überprüfbare Prinzipien — eine aufgeblähte Verfassung verbrennt nur Kontext und wird ignoriert.

So könnte ein fertiges Kurs-Template aussehen (es bringt auch die Schutzmechanismen von oben an einem festen Ort unter):

VORLAGE — constitution.md (Kurs: KI-gestütztes Programmieren)

```
# Projektprinzipien

## Technik
- Vanilla HTML, CSS, JavaScript. Kein Build-Step, kein Framework ohne Rücksprache.
- Die App läuft als statische Datei (index.html), ohne Server.
- Externe Bibliotheken nur mit kurzer Begründung in plan.md.

## Qualität
- Test-Driven: zu jeder Funktion mindestens ein Test, der zuerst fehlschlägt.
- Code ist dort kommentiert, wo die Absicht nicht offensichtlich ist.
- Benennung konsistent (Deutsch oder Englisch, aber nicht gemischt).

## Layout & UX
- Per Maus und Tastatur bedienbar, ausreichende Kontraste.
- Fehlermeldungen für Lernende verständlich formuliert.

## Arbeitsweise des Agenten
- Bei zweimaligem Fehlschlag desselben Tests: STOPP und Rückfrage (kein dritter Versuch).
- Vor einem Fix: vermutete Ursache und Plan nennen.
- Niemals das gesamte Repository ungefragt einlesen.
```

